purely data; they indicate what needs to be fulfilled, but they don't describe how that has to happen. For that, you need to write some handlers.

Creating authorization handlers to satisfy your requirements

Authorization handlers contain the logic of how a specific `IAuthorizationRequirement` can be satisfied. When executed, a handler can do one of three things:

- Mark the requirement handling as a success.
- Do nothing.
- Explicitly fail the requirement.

Handlers should implement `AuthorizationHandler<T>`, where `T` is the type of requirement they handle. For example, the following listing shows a handler for `AllowedInLoungeRequirement` that checks whether the user has a claim called `FrequentFlyerClass` with a value of `Gold`.

Listing 24.8 `FrequentFlyerHandler` for `AllowedInLoungeRequirement`

```
public class FrequentFlyerHandler :
    AuthorizationHandler<AllowedInLoungeRequirement>              ❶
{
    protected override Task HandleRequirementAsync(               ❷
        AuthorizationHandlerContext context,                      ❸
        AllowedInLoungeRequirement requirement)                   ❹
    {
        if(context.User.HasClaim("FrequentFlyerClass", "Gold"))   ❺
        {
            context.Succeed(requirement);                         ❻
        }
        return Task.CompletedTask;                                ❼
```